Future Interns

Date:12/03/2026

API Security Risk Analysis (Modern SaaS Skill)

This report aims to evaluate key security aspects such as authentication, authorization, data exposure, and request handling mechanisms. By analyzing these components, the report highlights common API security risks and explains their potential impact in a real-world business environment.

Email:syamnithin2003@gmail.com

Created & Presented By:
Yanamadala
Syam Nithin

Introduction

Modern applications rely heavily on Application Programming Interfaces (APIs) to enable communication between different systems, including mobile applications, web platforms, and third-party services. APIs play a critical role in delivering data and functionality in real time, making them a fundamental component of modern software architecture, especially in SaaS environments.

However, the widespread use of APIs also introduces significant security risks. If APIs are not properly secured, they can become a primary entry point for attackers, leading to unauthorized data access, data breaches, service abuse, and system compromise. Common vulnerabilities include weak authentication, improper authorization, excessive data exposure, and lack of rate limiting.

This report presents a read-only API Security Risk Analysis conducted on a public test API. The objective is to identify potential security weaknesses, assess their impact in a business context, and recommend practical remediation strategies. The analysis follows ethical guidelines and focuses on observation and evaluation rather than exploitation.

Scope & Ethics

This analysis was conducted under strict ethical guidelines to ensure no harm to systems, data, or services. The testing was limited to safe and controlled environments using public APIs.

Allowed

- Testing public or demo APIs
- Read-only requests (GET, safe POST where allowed)
- Documentation-based analysis
- Header, token, and response inspection

Not Allowed

- Exploitation or bypass attempts
- Flooding / DoS testing
- Attacking private or production APIs

Always stay ethical and legal.

Tools Used

1. **Postman**

2. **Browser DevTools**

Public API Tested

API Name: JSONPlaceholder
URL: https://jsonplaceholder.typicode.com

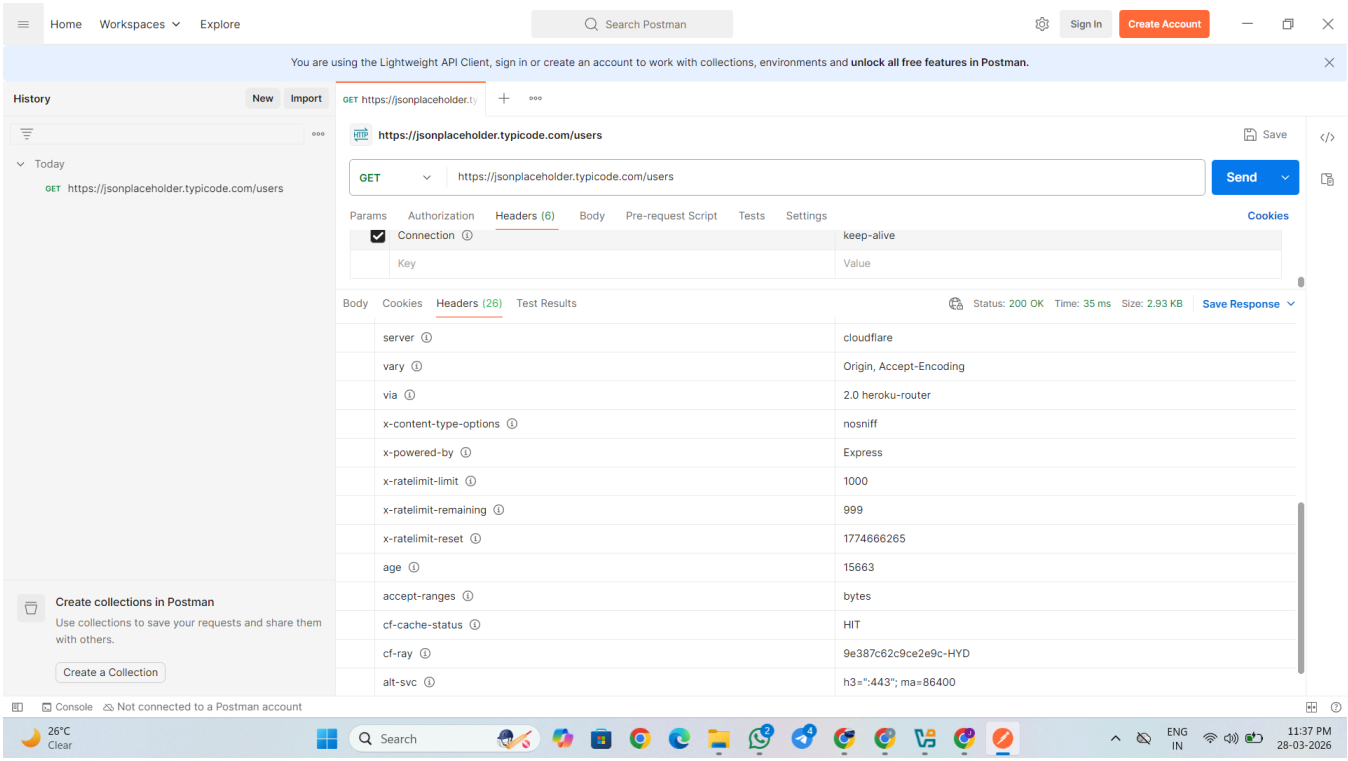
Postman

Step 1: Open Postman

Step 2: Set the Method as ‘GET’

Step 3: Paste the URL

Step 4: Click the Send



Successful API Response

- **Status:** `200 OK`
- This means the API request was successful and data is publicly accessible.

write:

The API returned a successful response (200 OK), indicating that the endpoint is publicly accessible without restrictions.

No Authentication Required

- There is no Authorization tab used
- No API key / token required

Risk Identified: **HIGH**

The API allows unrestricted access without authentication, which can lead to unauthorized data exposure.

Headers Analysis

Important headers visible:

- `x-powered-by: Express`
- `server: cloudflare`
- `x-content-type-options: nosniff` (good security header)
- `x-ratelimit-limit: 1000`
- `x-ratelimit-remaining: 999`

What this means:

Good:

- Security header (`nosniff`) is present
- Rate limiting exists

Risk:

- Technology exposure (`Express`) → attackers know backend

Note:

The API exposes backend technology details through headers (Express), which may help attackers identify potential vulnerabilities.

Rate Limiting

- Limit: 1000 requests
- Remaining: 999

Interpretation:

- Rate limiting exists but is very high → can still be abused

Note:

Although rate limiting is implemented, the high threshold may still allow abuse or automated attacks.

Data Exposure Risk

- Endpoint `/users` returns full user data (name, email, etc.)

Risk: MEDIUM

The API returns complete user datasets without filtering, increasing the risk of excessive data exposure.

Browser DevTools

Step 1: Open Chrome or Firefox

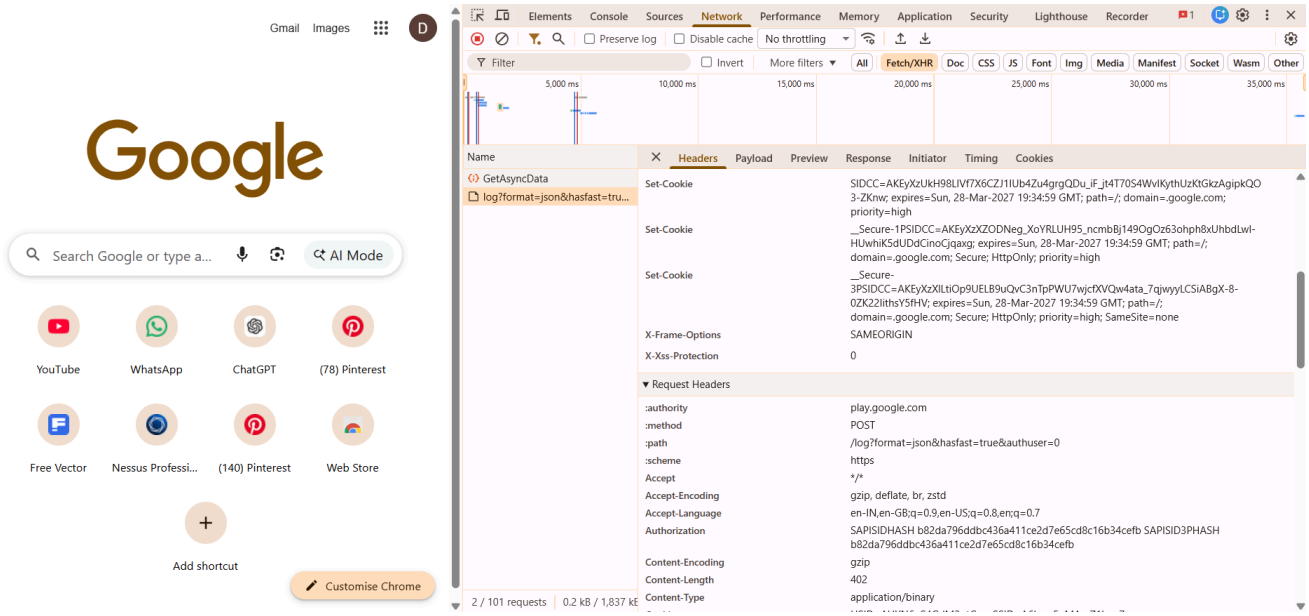
Step 2: Press F12 or Click on Inspect

Step 3: Click on Network and Press Ctrl+R for Refresh

Step 4: Click on Fetch/XHR

Step 5: Click on https://play.google.com/log?format=json&hasfast=true&authuser=0

Step 6: We can see the Headers



1. <u>Multiple Sensitive Cookies Set</u>

You can see:

- `SIDCC`
- `__Secure-1PSIDCC`
- `__Secure-3PSIDCC`

Meaning:

- These are session/authentication cookies
- Used for user tracking & authentication

<u>Security:</u> High

<u>Business Impact:</u>

- If exposed → **session hijacking risk**
- Attackers can impersonate users

Remediation:

- Always use:
  - `HttpOnly` (already present)
  - `Secure` (good)
  - `SameSite=Strict` (improve from `None`)

2. SameSite=None

- Cookie shows: `SameSite=None`

Meaning:

- Cookie is sent in cross-site requests

Security: High

Business Impact:

- Vulnerable to CSRF attacks

Remediation:

- Use:
  - `SameSite=Strict` (best)
  - or `Lax`

3. X-XSS-Protection Disabled

- `X-XSS-Protection: 0`

Meaning:

- Browser XSS protection is disabled

Security: Medium

Business Impact:

- Increased risk of **Cross-Site Scripting (XSS)**

Remediation:

- Enable protection OR
- Use strong **Content Security Policy (CSP)**

4. X-Frame-Options Present (Good Practice)

- `X-Frame-Options: SAMEORIGIN`

Meaning:

- Protects against **Clickjacking**

Security: No Issue (Good Security)

You can mention this as positive finding

5. Authorization Token in Request

- `Authorization: SAPISIDHASH ...`

Meaning:

- Token used for authentication

Security: Medium (Observation)

Business Impact:

- If leaked → unauthorized API access

Remediation:

- Keep tokens secure
- Use short expiry
- Avoid exposing in frontend

6. <u>Content-Type: application/binary</u>

<u>Meaning:</u>

- API response is binary encoded

<u>Security:</u> Low

<u>Business Impact:</u>

- Harder to inspect/debug
- Not standard for APIs

<u>Remediation:</u>

- Use JSON where possible

<u>Impact</u>

The identified API security risks can significantly impact a business by enabling unauthorized access to sensitive data due to the absence of authentication, leading to potential data breaches and privacy violations. Excessive data exposure further increases the chances of attackers misusing user information, while weak cookie settings and exposed tokens can result in session hijacking and user impersonation. Additionally, vulnerabilities like improper `SameSite` configuration and disabled XSS protection make the system prone to cross-site attacks such as CSRF and XSS. High rate limits allow automated abuse and data scraping, and exposure of backend technologies helps attackers plan targeted exploits. Overall, these issues can cause financial losses, legal consequences, and serious damage to an organization’s reputation and customer trust.

Conclusion

This API Security Risk Analysis highlights that even simple or public APIs can expose significant security vulnerabilities if not properly configured. Key issues such as missing authentication, excessive data exposure, weak cookie policies, and insufficient protection mechanisms can be easily exploited by attackers.

Although some good practices like security headers and rate limiting are present, they are not sufficient to ensure complete protection. Organizations must adopt a defense-in-depth approach, including strong authentication, proper authorization, secure cookie handling, and strict data filtering.

By implementing the recommended remediation strategies, businesses can significantly reduce security risks, protect user data, and build more secure and reliable applications.

END OF THE REPORT

THANK YOU FOR YOUR TIME AND ATTENTION

Email:syamnithin2003@gmail.com